

MPU-6500 + ESP8266 Tilt Mouse

Complete hardware, wiring, Wi-Fi/UDP, Ubuntu virtual-mouse, buttons, code and working explanation

Project owner: Nikhil Kushwah

Architecture: MPU-6500 → ESP8266 → Wi-Fi/UDP → Ubuntu Python → Linux virtual mouse.

This project converts controlled hand tilt into continuous mouse movement. It does not create a new game; it provides an input device that can control an existing desktop application or 2D game.

1. Final behavior

Input	Result
Tilt left	Cursor left
Tilt right	Cursor right
Tilt forward/backward	Cursor up/down
Small tilt	Slow movement
Large tilt	Fast movement
Hold tilt	Continuous movement
Return to neutral	Stops
Button 1	Mouse left click
Button 2	Reserved for custom action

2. Hardware and wiring

Hardware: NodeMCU ESP8266, MPU-6500, two momentary push buttons, breadboard/jumpers, Ubuntu laptop/PC and a common Wi-Fi network.

MPU-6500 → NodeMCU

MPU-6500	NodeMCU	Purpose
VCC	3V3	3.3 V supply
GND	GND	Common ground
SDA	D2 / GPIO4	I ² C data
SCL	D1 / GPIO5	I ² C clock
INT	Not connected	Not required

Buttons

NodeMCU	Connection	Logic
D5	Button 1 → GND	Pressed = LOW
D6	Button 2 → GND	Pressed = LOW

The buttons use INPUT_PULLUP. Each button is simply connected between its GPIO and GND; an external pull-up resistor is not required.

3. Sensor identification and verification

The tested I²C address is 0x68. The WHO_AM_I register returned 0x70, identifying the sensor as MPU-6500. Raw accelerometer registers produced changing values during movement. The final code therefore accesses MPU-6500 registers directly rather than using the MPU6050-specific initialization that failed during testing.

The final controller uses the accelerometer for tilt. Gravity provides the reference needed for a stationary tilted board; the gyroscope is not used by the mouse controller.

4. Tilt calculation and mapping

Python calculates two angles from AX, AY and AZ:

```
Xangle = atan2(AX, sqrt(AY2 + AZ2))
Yangle = atan2(AY, sqrt(AX2 + AZ2))
angle_degrees = angle_radians × 180 / π
```

Xangle controls horizontal mouse movement and Yangle controls vertical movement. The tested orientation required the X direction to be inverted in the final Python code so physical left produces cursor-left.

5. Wi-Fi and UDP packet

In the tested setup, the laptop used 192.168.1.14, the NodeMCU used 192.168.1.15 and UDP port 4210 was used. If the laptop IP changes, update laptopIP in the ESP8266 sketch.

The final packet format is:

```
AX, AY, AZ, BUTTON1, BUTTON2
Example: 1200, -500, 16000, 1, 0
```

Button values are transmitted as 1 when pressed and 0 when released.

6. Final ESP8266 code

Replace the Wi-Fi placeholders locally. Never publish your actual Wi-Fi password in a public repository.

```
#include <Wire.h>
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>

#define MPU_ADDR 0x68
#define BUTTON1 D5
#define BUTTON2 D6

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

const char* laptopIP = "192.168.1.14";
const unsigned int port = 4210;

WiFiUDP udp;

int16_t read16(byte reg)
{
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(reg);
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, (uint8_t)2);

    int16_t value = ((int16_t)Wire.read() << 8);
    value |= Wire.read();
    return value;
}

void setup()
{
    Serial.begin(115200);
    Wire.begin(D2, D1);

    // Wake MPU-6500
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x6B);
    Wire.write(0x00);
    Wire.endTransmission();
    delay(100);

    pinMode(BUTTON1, INPUT_PULLUP);
    pinMode(BUTTON2, INPUT_PULLUP);

    Serial.println();
    Serial.println("MPU-6500 started!");

    WiFi.begin(ssid, password);
    Serial.print("Connecting to WiFi");

    while (WiFi.status() != WL_CONNECTED)
    {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("WiFi connected!");
```

```
    Serial.print("NodeMCU IP: ");
    Serial.println(WiFi.localIP());
}

void loop()
{
    int16_t ax = read16(0x3B);
    int16_t ay = read16(0x3D);
    int16_t az = read16(0x3F);

    int button1 = !digitalRead(BUTTON1);
    int button2 = !digitalRead(BUTTON2);

    String data =
        String(ax) + "," +
        String(ay) + "," +
        String(az) + "," +
        String(button1) + "," +
        String(button2);

    udp.beginPacket(laptopIP, port);
    udp.print(data);
    udp.endPacket();

    Serial.println(data);
    delay(20);
}
```

7. Final Ubuntu Python code

The Python program uses evdev/uinput to create a virtual mouse. On the tested Ubuntu system, /dev/uinput requires root access, so start the program with sudo.

```
import socket
import math
import time

from evdev import UInput, ecodes

UDP_PORT = 4210
DEAD_ZONE = 5.0
MAX_ANGLE = 45.0
MAX_SPEED = 25
UPDATE_TIME = 0.01

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("0.0.0.0", UDP_PORT))
sock.setblocking(False)

capabilities = {
    ecodes.EV_REL: [ecodes.REL_X, ecodes.REL_Y],
    ecodes.EV_KEY: [ecodes.BTN_LEFT]
}

mouse = UInput(capabilities, name="MPU-6500 Tilt Mouse")

def angle_to_speed(angle):
    absolute_angle = abs(angle)

    if absolute_angle <= DEAD_ZONE:
        return 0

    effective_angle = absolute_angle - DEAD_ZONE
    effective_max = MAX_ANGLE - DEAD_ZONE

    normalized = effective_angle / effective_max
    normalized = max(0.0, min(1.0, normalized))

    speed = (normalized ** 1.5) * MAX_SPEED

    if speed < 1:
        speed = 1

    return int(speed)

print()
print("=====")
print("      MPU-6500 TILT MOUSE")
print("=====")
print("UDP port:", UDP_PORT)
print("X axis : LEFT / RIGHT")
print("Y axis : UP / DOWN")
print("Button 1 : LEFT CLICK")
print("Button 2 : RESERVED")
print("Press CTRL+C to stop.")
print()

latest_data = None
previous_button1 = 0

try:
    while True:
        while True:
            try:
                data, address = sock.recvfrom(1024)
                latest_data = data.decode().strip()
            except BlockingIOError:
                break

        if latest_data is not None:
            try:
                values = list(map(int, latest_data.split(",")))

                if len(values) >= 5:
                    ax, ay, az = values[0], values[1], values[2]
                    button1, button2 = values[3], values[4]

                    x_angle = math.degrees(
                        math.atan2(
                            ax,
                            math.sqrt(ay * ay + az * az)
                        )
                    )

                    y_angle = math.degrees(
                        math.atan2(
                            ay,
                            math.sqrt(ax * ax + az * az)
                        )
                    )

                    x_speed = angle_to_speed(x_angle)
```

```

        y_speed = angle_to_speed(y_angle)

        mouse_x = 0
        mouse_y = 0

        # X is intentionally inverted for the tested
        # physical orientation.
        if x_angle > DEAD_ZONE:
            mouse_x = -x_speed
        elif x_angle < -DEAD_ZONE:
            mouse_x = x_speed

        if y_angle > DEAD_ZONE:
            mouse_y = -y_speed
        elif y_angle < -DEAD_ZONE:
            mouse_y = y_speed

        if mouse_x != 0:
            mouse.write(ecodes.EV_REL, ecodes.REL_X, mouse_x)

        if mouse_y != 0:
            mouse.write(ecodes.EV_REL, ecodes.REL_Y, mouse_y)

        # Button 1 = mouse left button
        if button1 != previous_button1:

            if button1 == 1:
                mouse.write(ecodes.EV_KEY, ecodes.BTN_LEFT, 1)
            else:
                mouse.write(ecodes.EV_KEY, ecodes.BTN_LEFT, 0)

            mouse.syn()
            previous_button1 = button1

        # Button 2 is intentionally reserved.
        if button2 == 1:
            pass

        mouse.syn()

    except (ValueError, IndexError):
        pass

    time.sleep(UPDATE_TIME)

except KeyboardInterrupt:
    print()
    print("Stopping MPU-6500 mouse...")

finally:
    mouse.close()
    sock.close()
    print("Mouse controller stopped.")

```

8. How the Python code works

- 1. Receive:** UDP listens on port 4210. The socket is non-blocking so old packets can be discarded and the newest measurement used.
- 2. Parse:** The packet becomes AX, AY, AZ, Button 1 and Button 2.
- 3. Calculate:** atan2 produces Xangle and Yangle from the accelerometer.
- 4. Dead zone:** $\pm 5^\circ$ is treated as neutral, reducing unwanted movement from small sensor noise or hand tremor.
- 5. Speed:** Tilt beyond the dead zone is normalized toward 45° . A power of 1.5 gives finer control at small angles and faster movement at larger angles.
- 6. Continuous movement:** The program sends relative REL_X/REL_Y events repeatedly while the hand remains tilted.
- 7. Left click:** Button 1 changes BTN_LEFT between pressed and released states, so it can also be held.
- 8. Button 2:** Its signal is already transported through the system but currently has no action; it can be assigned later.

9. Running the project

First upload the ESP8266 sketch. Then check that the NodeMCU connects to Wi-Fi and prints its IP address.

Syntax check:

```
/usr/bin/python3 -m py_compile /home/nikhil/Arduino/imu_Game_control/main.py
```

Start the controller:

```
sudo /usr/bin/python3 /home/nikhil/Arduino/imu_Game_control/main.py
```

If uinput has not been loaded:

10. Testing procedure

- Connect the MPU-6500 and buttons exactly as shown.
- Upload the ESP8266 code.
- Confirm Wi-Fi connection and the NodeMCU IP.
- Confirm laptopIP in the sketch matches the laptop's current IP.
- Run the Python program with sudo.
- Keep the controller near neutral and check that the cursor is stable.
- Tilt left/right slowly and verify horizontal direction.
- Tilt forward/backward slowly and verify vertical direction.
- Press and release Button 1 to verify left click.
- Button 2 should currently have no visible effect.

11. Tuning

DEAD_ZONE = 5.0: Increase it if the cursor jitters near neutral; decrease it if small tilts do not respond.

MAX_SPEED = 25: Increase for faster cursor movement; decrease for slower movement.

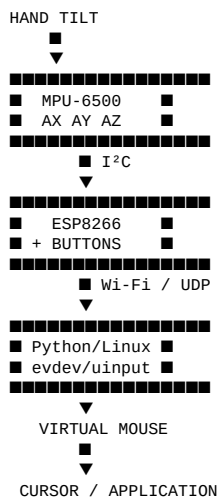
MAX_ANGLE = 45: Lowering this reaches maximum speed at a smaller tilt; increasing it requires more tilt.

The X-axis sign is intentionally reversed in the final code because the tested physical orientation initially made left tilt move the cursor right.

12. Troubleshooting

Problem	Check
uinput cannot be opened	Run with sudo; try sudo modprobe uinput.
No sensor data	Check 3.3V, GND, SDA=D2, SCL=D1 and I ² C address 0x68.
Cursor does not move	Check Wi-Fi, laptopIP, UDP port 4210 and that Python is running.
Horizontal direction wrong	Reverse the two X-axis assignments in Python.
Vertical direction wrong	Reverse the signs in the Y-axis section.
Cursor jitters	Increase DEAD_ZONE.
Cursor too slow	Increase MAX_SPEED.
Button 1 fails	Check D5-to-GND wiring and INPUT_PULLUP.
Button 2 does nothing	This is intentional; it is reserved for the future function.

13. Complete data-flow diagram



14. Final configuration

Item	Configuration
MPU address	0x68
SDA	D2 / GPIO4
SCL	D1 / GPIO5
Button 1	D5 → GND; mouse left click
Button 2	D6 → GND; reserved
UDP port	4210
Laptop IP in tested setup	192.168.1.14
NodeMCU IP in tested setup	192.168.1.15
Linux interface	evdev / uinput

Security note: Keep your real Wi-Fi credentials private. Use placeholders in GitHub/public documentation.